

C++ OOP — Խորը ուղեցույց

Հարցազրույցի ամբողջական C++ OOP presentation

Այս ուղեցույցը C++-ի OOP-ի խորը վերլուծություն է՝ 4 սյուները C++ syntax-ով, virtual function-ներ ու vtable, abstract class, operator overloading (բոլոր տեսակները շատ մանրամասն), composition vs inheritance, `this` pointer, constructor/destructor հերթականություն, `friend`, `static`, և C++-ին հատուկ OOP թեմաներ՝ ամբողջական կոդով և պատրաստի interview պատասխաններով:

1. Polymorphism (Բազմաձևություն)

Polymorphism-ը թույլ է տալիս Parent class-ի pointer/reference-ով աշխատել տարբեր Child class-երի object-ների հետ:

Runtime Polymorphism (Virtual Functions)

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() { cout << "Animal Sound\n"; }
    virtual ~Animal() {}
};

class Dog : public Animal {
public:
    void sound() override { cout << "Bark\n"; }
};

class Cat : public Animal {
public:
    void sound() override { cout << "Meow\n"; }
};

int main() {
    Animal* a1 = new Dog();
    Animal* a2 = new Cat();
    a1->sound();    // Bark
    a2->sound();    // Meow
    delete a1;
    delete a2;
}
```

Ինչու է պետք virtual

Առանց `virtual` -ի կապը compile-time-ում է (static binding), և կկանչվի pointer-ի տիպի մեթոդը.

```
class Animal { public: void sound() { cout << "Animal Sound"; } };
Animal* animal = new Dog();
animal->sound();    // "Animal Sound" (ՈՉ "Bark") -- static binding
```

`virtual` -ով կապը runtime-ում է (dynamic dispatch)' ըստ object-ի իրական տիպի -> "Bark":

Virtual Destructor (interview-ի սիրած հարց)

Երբ Parent pointer-ով delete ես անում Child object, destructor-ը պետք է virtual լինի, հակառակ դեպքում միայն Base-ի destructor-ը կկանչվի -> memory leak.

```
class Animal {
public:
    virtual ~Animal() { cout << "Animal destroyed\n"; }
};
class Dog : public Animal {
    int* data;
public:
    Dog() { data = new int[100]; }
    ~Dog() { delete[] data; cout << "Dog destroyed\n"; }
};

Animal* a = new Dog();
delete a;    // virtual-ի շնորհիվ -> Dog destroyed, հետո Animal destroyed
```

2. Compile-time Polymorphism

Function Overloading

Նույն անուն, տարբեր parameters. compiler-ը ընտրում է compile-time-ում.

```
class Calculator {
public:
    int add(int a, int b)          { return a + b; }
    int add(int a, int b, int c)   { return a + b + c; }
    double add(double a, double b) { return a + b; }
};
```

Operator Overloading (Հակիրճ ներածություն)

C++-ը թույլ է տալիս operator-ներին տալ իմաստ քո class-երի համար.

```
class Point {
public:
    int x, y;
    Point(int x, int y) : x(x), y(y) {}
    Point operator+(const Point& p) const {
        return Point(x + p.x, y + p.y);
    }
};

Point p3 = p1 + p2;    // p1.operator+(p2)
```

Սա էլ polymorphism-ի ձև է (նույն `+` symbol, տարբեր վարք): Ամբողջական operator overloading-ը՝ բաժին 7-ում:

3. Abstraction & Abstract Class

C++-ում abstract class-ը ստեղծվում է **pure virtual function**-ով (= 0).

```
class Animal {
public:
    virtual void sound() = 0;    // pure virtual -> class-ը abstract է
    virtual ~Animal() {}
};

// Animal a;    // ERROR: cannot instantiate abstract class

class Dog : public Animal {
public:
    void sound() override { cout << "Bark"; }
};

Animal* animal = new Dog();
animal->sound();    // քեզ չի հետաքրքրում Dog է թե Cat
```

Payment օրինակ (abstraction գործնականում)

```
class Payment {
public:
    virtual void pay(double amount) = 0;
    virtual ~Payment() {}
};

class CardPayment : public Payment {
public:
    void pay(double amount) override { cout << "Pay by card: " << amount; }
};

class PaypalPayment : public Payment {
public:
    void pay(double amount) override { cout << "Pay by paypal: " << amount; }
};

Payment* p = new CardPayment();
p->pay(100);    // օգտագործողին հետաքրքրում է միայն pay(), ոչ ներքին
```

4. Encapsulation C++-ում

Տվյալները `private`, հասանելիությունը՝ `getter/setter`-ով, `validation`-ով.

```
class BankAccount {
private:
    double balance = 0;
public:
    void deposit(double amount) {
        if (amount > 0) balance += amount;    // control + validation
    }
    bool withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            return true;
        }
        return false;
    }
    double getBalance() const { return balance; }
};
```

`const`-ը `getBalance()`-ի վրա երաշխավորում է, որ `getter`-ը `object`-ը չի փոփոխի:

5. Inheritance C++-ում & access

```
class Base {
private:    int a = 1;    // child չի տեսնում
protected: int b = 2;    // child տեսնում է
public:    int c = 3;    // բոլորը տեսնում են
};

class Derived : public Base {
public:
    void show() {
        // cout << a;    // ERROR (private)
        cout << b << c;    // OK
    }
};
```

Inheritance-ի access (C++-ին հատուկ, Java-ում չկա)

C++-ում ժառանգման ձևն ինքը ունի access.

```
class D1 : public Base    {};    // public    -> public մնում public, protected մնում protected
class D2 : protected Base {};    // protected-> public դառնում protected
class D3 : private Base   {};    // private    -> ամեն ինչ դառնում private
```

Հիմնականում օգտագործում են **public** inheritance (իրական «is-a»):

Constructor / Destructor հերթականություն

Interview trap. կառուցումը **base -> derived**, ոչնչացումը **հակառակ**.

```
class A { public: A(){cout<<"A ctor ";} ~A(){cout<<"A dtor ";} };
class B : public A { public: B(){cout<<"B ctor ";} ~B(){cout<<"B dtor ";} };

B obj;
// Output: A ctor B ctor ... B dtor A dtor
```


6. Composition vs Inheritance

Inheritance (IS-A)

```
class Animal {};  
class Dog : public Animal {}; // Dog IS-A Animal  
  
Animal* animal = new Dog(); // տրամաբանական
```

Composition (HAS-A)

```
class Engine {  
public:  
    void start() { cout << "Engine started"; }  
};  
  
class Car {  
private:  
    Engine engine; // Car HAS-A Engine  
public:  
    void startCar() { engine.start(); }  
};
```

Car-ը Engine չէ, Car-ը Engine ունի:

Strong Composition vs Aggregation

Strong Composition — մասը ամբողջի հետ է ապրում/մեռնում.

```
class Car {  
    Engine engine; // Car ջնջվեց -> Engine-ն էլ ջնջվում է  
};
```

Aggregation (weak) — մասը կարող է առանձին գոյություն ունենալ.

```

class Car {
    Engine* engine;                // արտաքին Engine
public:
    Car(Engine* e) : engine(e) {}   // Engine-ը դրսից է գալիս
};

```

Ինչու Composition — ճկունություն

```

class Engine {
public:
    virtual void start() = 0;
    virtual ~Engine() {}
};

class PetrolEngine : public Engine { public: void start() override { cout<<"Petrol"; } };
class ElectricEngine : public Engine { public: void start() override { cout<<"Electric"; } };

class Car {
    Engine* engine;
public:
    Car(Engine* e) : engine(e) {}
    void startCar() { engine->start(); }
};

Car tesla(new ElectricEngine());
Car bmw(new PetrolEngine());      // նույն Car, տարբեր Engine

```

Interview պատասխան

Favor Composition over Inheritance. composition-ը ավելի flexible է, ավելի քիչ coupling է ստեղծում, և ավելի հեշտ է maintain անել: Inheritance օգտագործիր միայն իրական "is-a" կապի դեպքում; "has-a"-ի դեպքում` composition:

7. Operator Overloading (շատ մանրամասն)

Operator overloading-ը թույլ է տալիս C++ operator-ներին տալ իմաստ քո class-երի համար: Interview-ում շատ սիրում են, որովհետև ստուգում է՝ հասկանում ես member vs non-member, return type, const-correctness, և pass-by-reference:

7.1 Ընդհանուր կանոններ

- Չի կարելի հորիսել նոր operator (օր. **)
- Չի կարելի փոխել arity-ն կամ precedence-ը
- Հնարավոր Չէ overload անել: :: . .* ?: sizeof
- Գոնե մեկ operand պետք է լինի user-defined type
- Պահպանիր սովորական իմաստը (+ պետք է գումարի, ոչ հանի)

7.2 Arithmetic operators (+, -, *, /)

Սովորաբար **const member** կամ **non-member**, վերադարձնում է **նոր object by value**.

```
class Point {
public:
    int x, y;
    Point(int x = 0, int y = 0) : x(x), y(y) {}

    Point operator+(const Point& p) const { // const: this-ը չի փոխվում
        return Point(x + p.x, y + p.y);    // նոր object
    }
    Point operator-(const Point& p) const {
        return Point(x - p.x, y - p.y);
    }
};

Point a(1, 2), b(3, 4);
Point c = a + b; // (4, 6)
```

7.3 Compound assignment (+=, -=)

Փոխում է **this**-ը, վերադարձնում **reference** (***this**) որ **chain-ը աշխատի** (**a += b += c**).

```
Point& operator+=(const Point& p) {
    x += p.x;
    y += p.y;
    return *this;    // reference to self
}
```

Լավ practice. `+`-ը կարելի է իրականացնել `+=`-ի միջոցով՝ կողի կրկնությունից խուսափելու:

7.4 Comparison operators (==, !=, <)

Վերադարձնում են `bool`, `const` են.

```
bool operator==(const Point& p) const {
    return x == p.x && y == p.y;
}
bool operator!=(const Point& p) const {
    return !(*this == p);    // reuse ==
}
bool operator<(const Point& p) const {
    return (x != p.x) ? x < p.x : y < p.y;    // sort-ի համար
}
```

C++20-ից կա `operator<=>` (spaceship), որ ավտոմատ գեներացնում է բոլոր համեմատությունները:

7.5 Stream operators (<<, >>) — ՊԱՐՏԱԴԻՐ non-member

`cout << p`-ում ձախ operand-ը `ostream` է (ոչ քո class-ը), ուստի member չի կարող լինել. պետք է **non-member**, հաճախ `friend`.

```

class Point {
    int x, y;
public:
    Point(int x = 0, int y = 0) : x(x), y(y) {}
    friend ostream& operator<<(ostream& os, const Point& p);
    friend istream& operator>>(istream& is, Point& p);
};

ostream& operator<<(ostream& os, const Point& p) {
    os << "(" << p.x << ", " << p.y << ")";
    return os;                // reference` chain-ի համար (cout << a << b)
}
istream& operator>>(istream& is, Point& p) {
    is >> p.x >> p.y;
    return is;
}

Point p(1, 2);
cout << p << endl;    // (1, 2)

```

Ինչու **friend**. որ non-member function-ը հասանելիություն ունենա **private** դաշտերին:

7.6 Subscript operator ([])

Վերադարձնում է **reference**, որ և կարդալ, և գրել կարողանաս (**arr[i] = 5**). սովորաբար երկու տարբերակ` const և non-const.

```

class Array {
    int data[100];
public:
    int& operator[](int i)          { return data[i]; }    // գրել/կարդալ
    const int& operator[](int i) const { return data[i]; } // const object-ի համար
};

Array arr;
arr[0] = 42;                // non-const -> գրում է
cout << arr[0];             // կարդում է

```

7.7 Increment / Decrement (++ , --) — prefix vs postfix

Postfix-ը ունի «dummy» **int** parameter` prefix-ից տարբերելու համար.

```

class Counter {
    int count;
public:
    Counter(int c = 0) : count(c) {}

    Counter& operator++() {           // prefix: ++c
        ++count;
        return *this;               // reference (արագ)
    }
    Counter operator++(int) {         // postfix: c++ (dummy int)
        Counter temp = *this;        // հին օբյեկտը պահիր
        ++count;
        return temp;                // by value (հին վիճակ)
    }
    int get() const { return count; }
};

```

Կարևոր. prefix-ը վերադարձնում է reference (`*this`), postfix-ը՝ հին object by value (նուսի ավելի թանկ):

7.8 Assignment operator (=) — Rule of Three

Pointer-ով class-ի համար պետք է self-assignment check և deep copy.

```

class MyString {
    char* data;
public:
    MyString& operator=(const MyString& other) {
        if (this == &other) return *this; // self-assignment պաշտպանություն
        delete[] data;                     // հին ազատիր
        data = new char[strlen(other.data) + 1];
        strcpy(data, other.data);          // deep copy
        return *this;
    }
};

```

7.9 Function call operator () — functor

Class-ը դարձնում է «կանչելի» (functor). STL-ում շատ օգտագործվող:

```

class Multiplier {
    int factor;
public:
    Multiplier(int f) : factor(f) {}
    int operator()(int x) const { return x * factor; }
};

Multiplier times3(3);
cout << times3(10);    // 30  (կանչում են object-ը ֆունկցիայի պես)

```

7.10 Member vs Non-member — կանոն

Member operator: `a.operator+(b)` -> ձախ operand-ը ՄԻՇՏ քո class-ն է
 Non-member operator: `operator+(a, b)` -> ձախ operand-ը կարող է լինել այլ տիպ

Member արա: `= [] ()` -> (պարտադիր member)

Non-member արա: `<< >>` (ձախը stream է),
 և commutative operator-ներ (`2 + point`), որ ձախ operand-ը այլ տիպ լինի

8. `this`, `pointer`, `static`, `friend`

this pointer

Ամեն non-static մեթոդ ստանում է `this`` ընթացիկ object-ի հասցեն.

```
class Point {
    int x;
public:
    void setX(int x) { this->x = x; }    // parameter vs member տարբերելու
    Point& self() { return *this; }      // chaining-ի համար
};
```

static member & method

`static` member-ը պատկանում է class-ին (ոչ object-ին)՝ մեկ instance բոլորի համար.

```
class Counter {
public:
    static int count;                    // declaration
    Counter() { count++; }
    static int getCount() { return count; }    // static մեթոդ, this չունի
};

int Counter::count = 0;                 // definition (class-ից դուրս)

// Counter::getCount() -> object պետք չէ
```

friend

`friend` function/class-ը հասանելիություն ունի `private` անդամներին (խախտում է encapsulation-ը վերահսկված ձևով, օր. `operator<<` -ի համար):

9. C++ vs Java — OOP տարբերություններ

	C++	Java
Multiple inheritance	class-ներով` այն (virtual)	միայն interface
Virtual by default	ոչ (պետք է <code>virtual</code>)	այն (բոլոր non-static)
Operator overloading	այն	ոչ
Destructor	այն (<code>~Class</code>)	ոչ (GC + finalize)
Memory	ձեռքով (<code>new/delete</code>)	Garbage Collector
Inheritance access	public/protected/private	միայն public
Pointers	այն	ոչ (references only)

10. Interview Short Answers

Runtime vs compile-time polymorphism?

Runtime = virtual functions (overriding), որոշվում runtime-ում; Compile-time = function/operator overloading, որոշվում compile-time-ում.

Ինչու virtual destructor?

Որ base pointer-ով delete անելիս derived destructor-ն էլ կանչվի, հակառակ դեպքում memory leak.

Ի՞նչ է pure virtual function?

`virtual void f() = 0;` — implementation չունի, class-ը դարձնում abstract, derived-ը պարտավոր է override անել.

Operator overloading` member vs non-member?

`<< >>` -ը non-member (ձախ operand-ը stream է); `= [] () ->` -ը պարտադիր member; մնացածը կախված է` commutative-ի համար non-member.

Prefix vs postfix ++ տարբերությունը?

Prefix (`++x`) reference է վերադարձնում; postfix (`x++`) ունի dummy `int` parameter և վերադարձնում է հին արժեքը by value (ավելի թանկ).

Composition vs Inheritance?

Inheritance = is-a; Composition = has-a. Favor composition over inheritance` ճկունության և քիչ coupling-ի համար.

Rule of Three?

Եթե class-ը պետք ունի destructor, copy constructor, կամ copy assignment-ից մեկը (սովորաբար raw pointer-ի պատճառով), ապա հավանաբար պետք ունի երեքն էլ.

Cheat Sheet

4 PILLARS (C++):

Encapsulation -> private + getter/setter (const)
Inheritance -> class D : public Base (is-a)
Polymorphism -> virtual (runtime) / overload (compile)
Abstraction -> pure virtual = 0 (abstract class)

VIRTUAL:

virtual մեթոդ -> dynamic dispatch (runtime)
virtual ~Base() -> ՊՈՐՏԱԴՐՈՐ polymorphic base-ի համար
= 0 -> pure virtual -> abstract class

OPERATOR OVERLOADING:

+ - * / -> const member, նոր object by value
+= -= -> reference (*this)
== != < -> bool, const
<< >> -> non-member friend, ostream&/istream& return
[] -> reference (const + non-const)
++ -- -> prefix: &(*this) | postfix: int dummy, հին by value
= -> self-check + deep copy (Rule of Three)
() -> functor

RELATIONSHIPS:

is-a -> Inheritance (Dog : Animal)
has-a -> Composition (Car has Engine) -> FAVOR THIS

ctor order: Base -> Derived | dtor order: Derived -> Base